Personalized PageRank for MovieLens Top 10 Retrieval

Student: Raimova Alina | **Team:** Individual
Email: aaraimova@edu.hse.ru | **Date:** 2026-10-27
Assignment: Graph-Based Recommendations with Personalized PageRank (Week 6, HW6)
Github link: [MOCK - repository URL to add]

Abstract

This study tests whether Personalized PageRank can improve warm-user movie retrieval over popularity, global PageRank, and a matrix-factorization baseline on MovieLens 100K. The protocol uses 79,619 training, 9,596 validation, and 10,785 test ratings, mean training rating μ = 3.58060262, and popularity Recall@10 = 0.07602291. With alpha selected on validation, PPR reaches test Recall@10 of 0.11218853 across 861 common users, compared with 0.03124894 for the corrected MF baseline. These offline results support testing PPR as the warm-user Top-10 recommender, while leaving viewing and business outcomes to a user-level online experiment.

Index Terms - personalized PageRank, movie recommendation, matrix factorization, retrieval evaluation

1. Introduction

Problem statement. Rank unseen movies from explicit ratings, treating missing ratings as unobserved, and compare popularity, global PageRank, PPR and MF under the same retrieval protocol.

Motivation. Low rating-prediction error does not guarantee useful Top-10 recommendations. Fair comparison requires common candidates, exclusions and evaluation users.

Concrete example. User 1 has not rated Fargo in training or validation. PPR ranks it first, and its held-out test rating of 5 makes it a hit.

Contributions.

- Implement and verify PageRank/PPR with equal outgoing probabilities, personalized restart and dangling-mass handling.
- Compare four methods, trace one recommendation numerically, and diagnose and fix MF regularization.

2. Related Work

MovieLens provides explicit ratings and timestamps [1]. PageRank redistributes probability through links [2]; personalized restart makes the distribution depend on a selected user [3]. MF models observed ratings through latent factors [4]. TensorFlow.js provides the tensor operations and memory management used here [5].

[1] F. M. Harper and J. A. Konstan, “The MovieLens Datasets: History and Context,” *ACM Transactions on Interactive Intelligent Systems*, vol. 5, no. 4, Art. 19, 2015, doi: 10.1145/2827872.

[2] L. Page, S. Brin, R. Motwani, and T. Winograd, “The PageRank Citation Ranking: Bringing Order to the Web,” Stanford InfoLab, Technical Report, 1999.

[3] G. Jeh and J. Widom, “Scaling personalized web search,” in *Proc. 12th Int. Conf. World Wide Web*, 2003, pp. 271-279, doi: 10.1145/775152.775191.

[4] Y. Koren, R. Bell, and C. Volinsky, “Matrix Factorization Techniques for Recommender Systems,” *Computer*, vol. 42, no. 8, 2009, doi: 10.1109/MC.2009.263.

[5] TensorFlow, “Tensors and operations,” *TensorFlow.js Guide*, and *TensorFlow.js API*, version 4.22.0. Accessed: Oct. 27, 2026.

Alternatives considered. Popularity and global PageRank provide nonpersonal baselines. MF provides an observed-rating predictor whose validation objective differs from PPR’s retrieval objective.

3. Method

Approach. Build a binary bidirectional user-movie graph from training ratings ≥4. For a node with degree d, each outgoing link has probability 1/d; dangling rows are zero. Starting with p_0=v, update

$$p(t+1) = \alpha[Q^T p(t) + d(t)v] + (1-\alpha)v,$$

where Q contains outgoing probabilities and d(t) is total dangling mass. Global PageRank uses uniform restart v; PPR restarts at the chosen user. Thus α follows links and 1−α restarts. Popularity counts distinct users with positive training ratings.

Pipeline.

1. Load ratings and movie metadata; split each user’s ratings chronologically.
2. Build the training graph and train MF on observed ratings.
3. Select α on validation and restore MF’s best validation-RMSE checkpoint.
4. Exclude previously rated movies, rank candidates and evaluate the common test cohort.

Tools and libraries. TensorFlow.js 4.22.0 runs Float32 calculations on WebGL; d3 7.9.0 displays the graph. Python 3.12.14 and NumPy 2.3.5 support independent numerical audits. Tensor scopes and explicit disposal manage memory; cooperative yields keep batch computation responsive. Hardware details and isolated algorithm timings were not recorded.

Key design decisions.

- The training graph contains 943 users and 1,611 movies: 2,554 nodes and 45,602 positive edges. One user and 253 movies have no positive links. Missing ratings are never zero-valued targets.
- Stop when the L1 residual is below 10^{-6}, with a 500-iteration cap. Batched PPR uses 32 columns with independent stopping. A finite-iteration partition correction controls Float32 mass drift.

Configuration. Table 1. Data and popularity baseline before model metrics. Source names identify the calculations; the essential values and settings are included here.

Quantity	Value	Calculation and settings
Train / validation / test ratings	79,619 / 9,596 / 10,785	`data.js`, `splitByTimestamp()`: per user, timestamp then movie ID; floor(80%), floor(10%), remainder; minimum 5 ratings
Training mean μ	3.58060261998 = 285,084 / 79,619	`script.js`, `hw6TrainMf()`: mean of all observed training ratings
Popularity test Recall@10	0.07602291344	`script.js`, `recallAtK("popularity",10,"test")`: relevance ≥4, training+validation exclusions, 861 users

MovieLens contains 100,000 ratings, 943 users and 1,682 metadata movies; no users are skipped. Sparsity is 93.6953% over the unpadded user-movie rectangle. The split is chronological within users, rather than globally chronological.

MF predicts $\mu+p_u^T q_i$ using 16 factors, seed 20261013, Adam learning rate 0.01, batch size 512, 30 epochs and L2 coefficient 0.05. A completed Week-5 checkpoint was unavailable, so the implemented standalone baseline is a deviation from requested reuse. PPR selects α from {0.50, 0.70, 0.85, 0.95}. Its probability scores and MF's rating predictions have different scales.

4. Experiments

4.1 Validation selection and final evaluation

Setup. Candidates are the 1,611 training movies. Validation excludes all training-rated movies; test excludes all training- and validation-rated movies, regardless of rating. Relevant held-out ratings are ≥4 and eligible. Rank by descending score, breaking ties by ascending movie ID. Recall@10 averages each user's hits divided by eligible relevant movies.

The shared warm cohort contains 846 validation users and 861 test users. Eighty-two users lack eligible relevant test movies, including the isolated user; 13 relevant test observations fall outside the candidate pool. MF is scoreable on the same candidates.

Results. Table 2. Validation sweep; all runs converge. Select α=0.50, using smaller α for exact ties.

α	Validation Recall@10	Maximum iterations
0.50	0.11448316	22
0.70	0.10479695	42
0.85	0.09002077	92
0.95	0.07375030	287

Table 3. Final test comparison on the same 861 users. Mean popularity counts positive-training users per recommended movie.

Method	α	Test Recall@10	Mean popularity
Popularity	-	0.07602291	272.00
Global PageRank	0.50	0.07969324	257.82
PPR	0.50	0.11218853	244.21
Standalone MF	-	0.03124894	94.31

Global PageRank uses the PPR-selected α and was not separately optimized. MF uses the earliest best-validation-RMSE checkpoint, epoch 4, with displayed validation RMSE 0.9779. Neither model was refit on test. PPR has the highest observed recall; MF's lower popularity does not establish better recommendations.

4.2 Fixed-user lists and controlled PPR demonstrations

Setup. Fix A=1 and B=2; both belong to the common test cohort. User 1 has 217 training ratings, 131 positive, and 19 eligible relevant test movies. User 2 has 49 training ratings, 32 positive, and two eligible relevant test movies. Generate eight final Top-10 lists using the selected α and standard test filtering; missing held-out ratings remain unknown.

Results. Table 4. Final lists: mean popularity and the number of shared movies between users.

Method	User 1 mean popularity	User 2 mean popularity	Shared / 10
Popularity	246.8	262.1	1
Global PageRank	239.4	247.4	1
PPR	228.4	226.7	1
MF	99.1	79.9	4

PPR starts with *Fargo* (Crime, Drama, Thriller) for user 1 and *Return of the Jedi* (Action, Adventure, Romance, Sci-Fi, War) for user 2. MF starts with *The Wrong Trousers* (Animation, Comedy) and *Persuasion* (Romance), respectively. User 2 has no Top-10 hits under any method.

Table 5. Demonstrations using the same training graph, kept separate from final evaluation.

Setting	User	α	Mean popularity	Overlap with setting 1
1	1	0.85	245.5	-
2	2	0.85	254.9	0/10
3	1	0.50	228.4	7/10

Changing the user at α=0.85 gives zero overlap, partly because exclusions differ: all ten user-2 recommendations were already rated by user 1. Lowering α for user 1 preserves seven movies: *Scream*, *Air Force One* and *Titanic* leave; *Trainspotting*, *Sense and Sensibility* and *E.T.* enter. Higher α produces the more popular list in this example; this is not a universal quality or popularity rule.

4.3 Worked example: user 1, Fargo, α=0.50

Setup. Fargo (movie 100) has 377 positive-training neighbours. In iteration 21, the final L1 residual is 9.95331675×10^{-7}. Incoming mass is the sum of neighbouring probabilities divided by their degrees. Fargo receives no direct restart contribution because restart is at user 1; total dangling mass is zero. The update and correction are implemented in `graph.js`, including `graphCorrectPartitionMass()`.

Results. Table 6. Actual final-iteration Float32 values.

Component	Value
Incoming mass at Fargo	0.0013710429193452
Dangling contribution	0
Restart contribution	0
Raw score: 0.50 × incoming	0.0006855214596726
Raw movie-partition mass	0.3333335220813751
Target movie-partition mass	0.33333349227905273
Final score	0.0006855214014649391
Difference from saved score	0

The numerical correction is Float32(Float32(raw / 0.3333335220813751) × 0.33333349227905273). It reduces the raw score by 5.82076609×10^{-11}. The user-partition target at iteration t is $(1+\alpha(-\alpha)^t)/(1+\alpha)$; the movie target is its complement. Original and observed batch outputs match exactly. An independent Float64 solver gives 0.0006855209963030 16, differing by $-4.05161923\times10^{-10}$ through numerical precision.

Fargo is absent from user 1's training and validation histories, is eligible, and ranks **1st of 1,367 candidates**. Its held-out test rating in `u.data` is **5**. The three Top-10 hits are:

Rank	Movie ID	Title	Test rating
1	100	Fargo (1996)	5
2	258	Contact (1997)	5
7	222	Star Trek: First Contact (1996)	4

The 19 eligible relevant test IDs are 6, 18, 20, 32, 100, 111, 129, 154, 169, 171, 178, 209, 221, 222, 228, 242, 256, 258 and 270. The remaining nine test ratings are below 4. Therefore user-1 **Recall@10 = 3/19 = 0.1578947**, distinct from cohort recall. Filtering, ranking and recall use `script.js`, including `recallAtK()`.

4.4 Implementation verification

Setup. `runSelfChecks()` and `hw6CheckBatchedPpr()` cover transition normalization, restart, dangling nodes, probability mass, convergence, candidate filtering, macro recall, batch parity, tensor cleanup and the MF-loss regression.

Results. The recorded post-fix harness reports **21 PASS, 0 FAIL, 0 PENDING**. Twelve user/α fixture cases match single-user PPR. Six repeated propagation runs retain 2/2 live tensors with MF loaded; the separate Fargo trace retains 0/0. All four validation sweeps converge within the cap.

5. Discussion

Failure case. MF initially produced nearly identical predictions close to the global mean μ, poorly distinguishing users and movies. It was fixed.

Root cause. MSE was averaged over the current mini-batch, while L2 was summed over all user and movie factors on every batch. At the configured coefficient, this imbalance suppressed factor learning.

Fix and verification. `hw6MfBatchLoss()` now averages L2 over factor rows gathered for the current batch, retaining repeated-observation weights. All reported final MF metrics use the corrected model.

What worked. PPR worked best in our tests and found more movies that users had rated highly. Based on this, I would test PPR as the main recommendation method instead of our current MF model. We still need to check how it performs with real users before making the switch. This does not mean that PPR is always better than MF.

What surprised me or did not work. I was surprised that MF still performed poorly after the fix. It recommended less popular movies, but this did not make its recommendations more relevant.

6. AI Usage Disclosure

- **AI tools used:** OpenAI Codex with browser/terminal tools.
- **How AI was used:** Implementation, debugging, numerical checks, and report drafting.
- **What I personally verified:** Main logic for alghoritms, experiment's results, app stability.
- **What I trusted without full verification:** implementation of alghoritms and changes in the front of the app.
- **Session log reference:** in the file `raimova_alina_hw6_session.json`.

References

See §2 (numbered [1]-[5]).